RELAYWORKS / BOOK 02

The conversation and demonstration book

The Business Owner Playbook

Ask better questions. Show the proof. Define useful work.

Your conversation, demonstration, and follow-through checklist for Relayworks.

This is the book for explaining the work to another person. **The Works** teaches the exact clicks. This companion teaches what to ask, what to show, what each app proves, how to handle objections, and how to leave with a useful next step. Read it like a field guide, then use the checklists and notes during rehearsal.

The strongest claim you can make is a specific one you can demonstrate. “We entered this address once, saved it in these three independent systems, checked the saved values, and recovered this interrupted handoff” is stronger than “we can automate anything.”

Prepared for this workspace · September 24, 2026 · All practice businesses and people are fictional.

Contents

00. Set up your conversation book
01. Your identity and a useful opening
02. Discovery: find the repeated work and its exceptions
03. Turn the story into a process worth showing
04. Talk about value without inventing savings
05. Run a meeting that the owner can follow
06. HVAC and field service: a call becomes coordinated work
07. Law office: intake, consistent documents, dated tasks
08. Clinic administration: registration and appointment coordination
09. Retail and inventory: count once, apply the receipt once
10. Independent sellers: one item, prepared listing, one sale
11. Property operations: request, contractor, owner, invoice match
12. Community and events: registration, capacity, badges, attendance
13. Sports and personal reports: show where the facts come from
14. The tools conversation: messy files, sheets, PDFs, and scans
15. The process conversation: maps, APIs, records, timers, and proof
16. Real access: what is connected, what needs approval
17. Free demonstration, real operating costs, and scale
18. Objections and practical responses
19. The senior-team roles across the SDLC
20. Scope a complete first business workflow
21. Follow through after the demonstration
22. Your reusable meeting and readiness checklist

CHAPTER 00 · 6 CHECKPOINTS

Set up your conversation book

- [ ] Open **THE_WORKS.html** in another tab. Learn its startup chapter before practicing a sales conversation.
- [ ] Use **Contents**, **Search this book**, and **Next chapter** to navigate. **Read whole book** gives you a continuous document.
- [ ] Tick the actions you have practiced. A tick is your learning record; it does not start the software or send a message.
- [ ] Write your own examples in the notes boxes. Keep customer secrets and sensitive records out of these browser notes. Use fictional names while rehearsing.
- [ ] Use **Export progress** to back up your checkmarks and notes. Import that JSON if you switch browsers or move between the file edition and the local web edition.
- [ ] Use **Print / save PDF** for a marked-up copy, or open the supplied blank PDF in `books/`.

Choose a route through the book

Chapters 01–05 teach the conversation. Chapters 06–13 are the eight business demonstrations. Chapters 14–17 explain the tools, real access, cost, and capacity. Chapters 18–22 cover objections, delivery roles, scoping, follow-through, and your reusable meeting worksheet.

You do not need to show all eight businesses to one owner. Learn the whole station; choose one story that resembles that owner's work. A plumbing company can understand the HVAC demonstration without pretending it is already connected to its own software.

Your notes

My first audience, the workflow I will lead with, and the questions I need to practice asking naturally.

_____

_____

_____

CHAPTER 01 · 5 CHECKPOINTS

Your identity and a useful opening

What you are offering

You are demonstrating a software engineering capability: understanding a manual process, connecting permitted sources and destinations, preserving business rules, and making the result inspectable. The showcase represents several engineering disciplines. It does not establish a real headcount, employment history, certifications, customer list, or partnership with a vendor.

Use “I” when you are presenting your own work. Use “we” for work your actual team will deliver. Explain the engineering roles in chapter 19 without inventing people or credentials.

A 30-second introduction

“I build the connections between the tools a business already uses. If a name, address, order, or job gets copied between a spreadsheet, scheduling screen, and paperwork, we can examine that handoff. This demonstration lets you do the copying yourself, then enter the facts once and inspect the saved result in three separate apps. I also show what happens when a destination refuses access or goes offline.”

Pause. Let the owner respond. Your next sentence should relate to what they say, rather than reciting your whole feature list.

An invitation that earns a useful answer

“Think about the last job or order that made someone type the same information twice. Where did it arrive, and where did it have to end up?”

If they cannot think of one, narrow the question: “When a new customer calls, does that person's name get entered in both your schedule and your billing system?” If they still do not have a repeated handoff, do not manufacture a problem. Explore document preparation, reporting, inventory, or another process only if it matters to them.

- [ ] Practice the introduction aloud without saying “AI,” “API,” or “integration platform” before describing the actual problem.
- [ ] Replace “we automate everything” with the one handoff you can prove today.
- [ ] Ask the opening question and stop talking long enough to hear the answer.
- [ ] Repeat their process back in their words: “The request arrives in ___; ___ retypes it into ___; then ___ needs ___.”
- [ ] Ask whether that description is accurate before opening the demonstration.

Your notes

My natural introduction, the owner's words for the problem, and the sentence I will use to connect it to a demonstration.

_____

_____

_____

CHAPTER 02 · 6 CHECKPOINTS

Discovery: find the repeated work and its exceptions

Start with one recent example

An owner may say “our office is chaos” or “everything is in Excel.” Those are starting points, not specifications. Walk through a single recent job using a redacted or fictional example. Identify the person, screen, source, destination, and decision at each step.

Ask	Why it matters	What to write down
What starts the process?	Defines the trigger.	Call, submitted form, file, receipt, sale, or timed report.
Who touches it first?	Identifies the operator.	Role and normal working hours.
Where is the original information?	Identifies the source to read.	Named app, sheet tab, PDF, email attachment, or folder.
Which exact facts get retyped?	Reveals the mapping.	Field names, examples, and required formats.
Where does each fact go?	Identifies distinct destinations.	Customer screen, work order, draft invoice, report, etc.
What means it is the same customer or job?	Prevents duplicates and wrong matches.	Customer ID, job reference, SKU, receipt ID; names alone are weak keys.
What must a person approve?	Preserves decisions.	Slot, estimate, conflict review, publication, payment.
What happens when something is missing?	Defines a review path.	Who fixes it, how they are notified, and what must remain unsaved.
How do you know the handoff finished?	Defines verifiable success.	Saved record, linked ID, exact values, reconciled counts.
How often, and how many at the busiest time?	Defines workload and burst size.	Typical day, peak batch, seasonal exceptions.
What happens if the software is unavailable?	Reveals recovery needs.	Retry, pause, manual fallback, owner of the queue.
Who controls the account and permissions?	Identifies the access path.	Account administrator and approved test environment.

Questions that uncover the expensive exceptions

“What changed after you already entered it?” often reveals more than “what do you type?” A changed address, rescheduled appointment, corrected quantity, or canceled job can cause inconsistent records. Ask which system wins when two copies disagree.

“Show me the last mistake you had to repair” is useful if they can show it without exposing private information. If they cannot, have them describe the sequence with made-up names. You are looking for how the mistake propagates and how it is corrected.

- [ ] Separate facts copied unchanged from values calculated or approved later.
- [ ] Record which system owns each fact. For example, receiving owns counted quantity; a sales system owns reserved units.
- [ ] Identify the happy path, a missing-field case, a duplicate, a correction, and an unavailable destination.
- [ ] Ask how deleted or canceled records should be treated. This demo does not prove a universal deletion-sync policy.
- [ ] Find out whether a person currently checks completion or merely assumes it happened.
- [ ] Confirm the discovery summary before discussing a technical solution.

Your notes

Trigger → source → person → repeated fields → destinations → approvals → exceptions → completion proof. Use sample identifiers, not private data.

_____

_____

_____

CHAPTER 03 · 6 CHECKPOINTS

Turn the story into a process worth showing

Draw the before state

Use five simple boxes: **arrival → first entry → second entry → third entry → checking/correction**. Put a person's role above each manual step. Under each box, write which fields they type and what screen or document holds the result.

Now draw the after state: **arrival → extraction/review → mapping → authorized saves → read-back → exceptions to a person**. Keep business decisions visible. “A person approves the estimate” must not quietly become “the software invents the estimate.”

Match the owner's process to a demonstration

Owner's situation	Start here	Strongest second proof
Service calls, jobs, customer details, invoice drafts	Desert Air / HVAC	Complete a visit, then a failed billing handoff and resume.
Client intake and repeated document fields	Juniper Legal	Review hold plus a populated document.
Registration and appointment entry	Cedar Clinic	Conflicting appointment rejected.
Deliveries, counted stock, availability	Corner Supply	Replay a receipt without adding stock twice.
One-off items, listing preparation, packing	Second Story	Sell once, then reject another sale of the same item.
Repair requests, contractors, owner updates	Maple Properties	Hold an invoice that does not match the job or approved amount.
Attendees, capacity, badges, attendance	Gather Workshop	Waitlist/check-in rule and Google Sheet import.
Repeated reporting from public data	Touchline Daily	Show source URL, event date, retrieval time, and actual saved report.

Pick a small, complete demonstration

Small means one clear business event, not an unfinished interface. A complete proof includes the source, manual effort, transformed values, saved destinations, and at least one exception. If you show five disconnected screens without the same record moving between them, the owner cannot assess the result.

- [ ] Write the demonstration's one-sentence promise: “When ___ happens, ___ should appear in ___, with ___ checked.”

- [ ] Choose the primary business reference that will appear everywhere.
- [ ] Mark which values come from the source and which are defaults or deliberate choices.
- [ ] Choose the failure that matters to this owner: access, outage, double booking, duplicate receipt, oversell, or approval mismatch.
- [ ] Prepare a before/after screenshot pair, then show the live saved records as the main proof.
- [ ] Keep unrelated scenarios available for questions; do not make the owner sit through all eight by default.

Your notes

My before process, after process, key identifier, required human decision, and exception to demonstrate.

_____

_____

_____

CHAPTER 04 · 5 CHECKPOINTS

Talk about value without inventing savings

Distinguish counts, measurements, and estimates

Desert Air has 32 manual field positions across its three forms and 15 unique intake fields. Eight contact facts appear in all three destinations: 24 field positions representing eight facts. Those are structural counts in this demo. They do not establish that every field takes the same time, that an owner enters that many records, or that a particular business will save a claimed percentage.

Measure a real operator doing a representative task if the business permits it. Include finding the source, opening screens, typing, checking, correcting errors, and handling exceptions. Measure the automated route with review and recovery time included. Comparing only a fast API call with a whole manual task exaggerates the benefit.

A clearly hypothetical calculation

Suppose a business has 20 comparable jobs per day. Suppose measured manual handling is 6 minutes per job and the complete proposed route, including review and typical exceptions, averages 2 minutes. At 22 working days, the arithmetic is:

```
20 jobs/day × (6 − 2) minutes/job × 22 days = 1,760 minutes
1,760 ÷ 60 = about 29.3 hours of handling capacity per month
```

These numbers are an illustration, not observed Relayworks performance or a promised result. Recovered capacity can mean faster turnaround, less backlog, or more time for customers. It becomes cash savings only if the business can actually reduce a cost. If you multiply time by an hourly cost, label the result an estimate of labor capacity value and subtract ongoing operating/support costs separately.

A measurement worksheet

Measure	Before	After	Where the evidence comes from
Typical and peak volume	___	___	Counts over an agreed period.
Complete handling time per case	___	___	Comparable observed cases, including review.
Re-entry/correction rate	___	___	Clearly defined errors and sample size.
Time until all destinations are correct	___	___	Arrival and verified completion timestamps.

Measure	Before	After	Where the evidence comes from
Exceptions needing a person	___	___	Queue records and resolution time.
Ongoing tool/support cost	___	___	Actual selected deployment and vendor terms.

- [ ] Label every number as a field count, observation, estimate, or design target.
- [ ] Ask which outcome matters: speed, accuracy, visibility, capacity, or customer response.
- [ ] Include exceptions and review in the proposed time saving.
- [ ] Avoid predicting job reductions or guaranteed revenue from a short demonstration.
- [ ] Define a baseline and an acceptance target before promising an improvement.

Your notes

The owner's baseline, the metric they care about, assumptions that need measuring, and the smallest useful success target.

_____

_____

_____

CHAPTER 05 · 10 CHECKPOINTS

Run a meeting that the owner can follow

Before the meeting

- [ ] Start Relayworks using The Works chapter 01. Open this book in a separate tab.
- [ ] Stop any source schedule. Restore the chosen business's destinations to **Online** and **Access granted**.
- [ ] Reset only the business being rehearsed. Close stale saved-record tabs.
- [ ] Open **Do it manually**, the intake or ugly workbook, and the relevant before/after gallery.
- [ ] If a live public source is part of the demonstration, test it once. Prepare the labelled offline recording if the network is unavailable.
- [ ] Choose the owner-level explanation and the technical evidence you can open if asked.

Five minutes: a first conversation

Spend roughly one minute on their process, one minute showing repeated fields, two minutes showing one intake and three saved results, and one minute discussing the next useful question. These are rehearsal targets, not an obligation to rush the owner.

Say: “Here is the original information. Here is the same customer reference on three forms. Now I enter it once. These are the three saved records, with their own IDs. Which part resembles your office, and which part works differently?”

Fifteen minutes: a serious demonstration

Use the first three minutes for discovery. Show enough manual entry to establish the repeated work. Run the automation, inspect all three records, and show one business follow-up. Deliberately block a destination and recover it. End with the owner's actual software names, approval owner, and one measurable next step.

If time is short, show the retained manual screenshots and say they were recorded earlier. Do not suggest that a screenshot was created by the run you just performed.

Thirty minutes: workflow workshop

Spend the extra time on the owner's exceptions and source quality. Open the ugly workbook, show a flagged row, explain a matching key, and sketch the real before/after process. Inspect the HTTP trace if their technical person wants it. Agree on the unanswered access and business-rule questions before discussing a delivery commitment.

When the live source or demonstration fails

“The source is unavailable right now. This screen shows that it has not completed. I can use the saved recording to demonstrate the remaining steps, and it will be labelled as a replay.” If a destination fails, show what was saved, explain why it stopped, restore the simulated service if appropriate, and resume. An honest recovery can be more informative than a flawless run.

- [ ] Ask the owner to point to the field or step that matters most.
- [ ] Open a real saved record rather than stopping at a green banner.
- [ ] Ask “what would need to be different for your process?” after the proof.
- [ ] End with one agreed next step; do not pile on unrelated tools.

Your notes

My opening, primary workflow, exception, evidence to open, time available, and next-step question.

_____

_____

_____

CHAPTER 06 · 13 CHECKPOINTS

HVAC and field service: a call becomes coordinated work

The business story

An office takes a service call. Customer details belong in the customer record; the technician needs contact, equipment, issue, and slot; billing needs the correct customer and job reference. The same facts get copied because those screens serve different people.

Say: “The goal is that the dispatcher, technician, and billing person can each work in their own screen while using the same approved customer and job facts.” This is the clearest opening demo for almost any appointment-based service company.

App	What it owns in this demonstration	What to point out
Customer Profiles	Contact and address details, access notes, and links to service jobs.	One customer can retain its identity across later jobs.
Dispatch Desk	Job reference, issue, equipment, technician, visit slot, and completion details.	The job is a separate record linked to the customer.
Billing Desk	Job-linked diagnostic fee and the later invoice draft calculation.	Billing receives the correct identifiers and approved completion amounts.

Discovery questions

- [ ] Ask where calls first become written records: paper, spreadsheet, CRM, or a dispatch product.
- [ ] Ask who chooses the technician and slot, and what counts as a scheduling conflict.
- [ ] Ask whether billing starts from a work order, a technician's text, or a second entry screen.
- [ ] Ask how repeat customers, multiple properties, revised addresses, and canceled calls are identified.
- [ ] Ask which labor, parts, fees, and approvals are needed before an invoice may be finalized.

Proof sequence and words to use

- [ ] Follow The Works chapters 04–06. Show Avery's source paperwork and the three blank manual forms. Type the common contact fields into each so the repeated work is visible.

- [ ] Say: “These are 32 field positions across the forms, representing 15 unique intake facts. The point is repeated handling, not a measured time-saving claim.”
- [ ] Reset, open **Automate it**, preview the mapping, and click **Populate all three apps**.
- [ ] Open each saved destination. Point to `C-014`, `JOB-1042`, the same address, and the distinct record IDs.
- [ ] Click **Complete the visit**. Show 1.5 approved hours × $80, plus $35 parts, plus the $95 diagnostic fee: a $250 draft total.
- [ ] Say: “The totals here come from approved input. This creates a demonstration invoice draft; it has not charged a card, filed tax, or sent an invoice to a real customer.”
- [ ] Replay the completion and show that the identified event does not add charges again. If desired, enter a second job reference and retain the existing customer identity.
- [ ] Use The Works chapter 19 to interrupt Billing Desk access and resume after restoring it. Point out that the earlier saves are retained.

Technical follow-up and next step

The API maps one contact fact to different field names, saves through each workplace API, then retrieves the saved row to verify it. Customer and job matching keys differ deliberately. Ask for the real product names and whether their supported interfaces allow customers, jobs, appointment changes, and invoice drafts. ERPNext is a researched candidate, not the system you just demonstrated.

Close with: “Could we map one service request, one completion, and one corrected request using fictional data from your actual field layout?” Success would mean correct linked records, no duplicate charges on replay, and a visible exception when a required destination cannot accept the update.

Your notes

Their dispatch/billing tools, repeated fields, customer/job matching rules, approvals, and the first service workflow to examine.

_____

_____

_____

Law office: intake, consistent documents, dated tasks

The business story

A prospective client's contact details and matter description appear in a matter record, document packet, and appointment/task list. Repeated entry can create mismatched names or addresses. The useful automation copies reviewed facts into administrative destinations and keeps the review decision visible.

Say: “The person decides whether the information and next step are appropriate. The software carries those reviewed facts into the administrative records and prepared paperwork.”

App	What it owns	Proof to show
Matter Desk	Client contact, other party, matter reference, and description.	The same `MAT-0204` anchors the process.
Document Desk	Client/office facts used in a populated demonstration document.	Repeated names and addresses agree with the intake.
Task Calendar	Appointment and matter-linked task information.	It uses the reviewed appointment, rather than inventing a deadline.

- [ ] Ask which intake fields are retyped into matter software and document templates.
- [ ] Ask which facts need staff or lawyer review, including spelling, parties, matter classification, and intended appointment.
- [ ] Ask which template revisions are authoritative and who approves changes.
- [ ] Ask how they recognize an existing client or matter without using a person's name as the sole key.
- [ ] Ask how they currently record that a review occurred.

Demonstrate it

- [ ] Follow The Works chapter 13. Show the three manual forms: 7, 6, and 5 field positions.
- [ ] Reset and populate from the nine-field intake. Open Matter Desk, Document Desk, and Task Calendar.
- [ ] Download a populated document and point to the exact repeated facts. Describe it as the supplied demonstration template.

- [ ] Click **Demonstrate a review hold**. Show the case that requires explicit review before proceeding.
- [ ] Explain: “This demonstrates a rule that stops the handoff and asks a person to review it. It is not a comprehensive conflict search, a legal conclusion, a court deadline calculator, or a filing.”
- [ ] Show source provenance and the saved reference so the owner can see where the document values came from.

If the owner asks about their actual document system

Explain that connecting a real system requires its supported interface, approved account access, matching fields, and the office's approved templates and review rules. docassemble is a researched document-assembly candidate; it has not been installed as one of these three workplaces. The demonstration does not establish suitability for confidential client data.

Close with: “Which one template causes the most repeat typing, and could we examine a blank or fictional copy plus its approved source fields?” A useful acceptance test compares every populated field with reviewed source facts and confirms a held case produces no unapproved continuation.

Your notes

The repeated template, authoritative source, matter identifier, reviewer, hold conditions, and a fictional sample to request.

_____

_____

_____

CHAPTER 08 · 11 CHECKPOINTS

Clinic administration: registration and appointment coordination

The business story

Registration, scheduling, and intake paperwork need overlapping demographic information. Demonstrate that information flow with fictional patients. Keep this story about administrative work: it does not diagnose, triage, determine coverage, or provide treatment recommendations.

App	What it owns	Proof to show
Registration Desk	Patient reference and demographic/contact facts.	The identity remains visible even when a later step fails.
Appointment Board	Provider, slot, visit type, and patient link.	A conflicting provider/slot is rejected.
Intake Packet Desk	Demographic/referral facts used for the intake packet.	Paperwork is populated after the preceding handoffs succeed.

- [ ] Ask which demographic facts staff copy between registration, scheduling, and forms.
- [ ] Ask which record identifier is authoritative and how corrections are reviewed.
- [ ] Ask whether the scheduling system exposes supported booking operations and conflict responses.
- [ ] Ask who may view each field and who approves a proposed administrative integration.
- [ ] Ask how failed or partially completed registrations are worked today.

Demonstrate it

- [ ] Follow The Works chapter 14 using synthetic patient `PAT-0310`. Show 6, 7, and 7 manual field positions, then the ten-field intake.
- [ ] Populate and inspect the three independent records.
- [ ] Click **Try a conflicting appointment**. Show that a second synthetic patient cannot occupy the same provider/slot.
- [ ] Point out the partial result: the second registration may already exist while the appointment fails. A partial save is not the same as a completed visit booking.

- [ ] Correct the intended appointment to the free slot specified in The Works and submit the corrected intake. Inspect the new run and matching patient identity.
- [ ] Say: “This proves a local scheduling rule and transparent partial completion. It does not prove that this demo is ready to hold real patient information.”

Technical and production questions

OpenEMR is a researched candidate with documented APIs, not a connected system here. An actual deployment needs review of the exact product/version, supported operations, identity matching, account roles, environment, data handling, and the organization's approval process. Do not claim a certification or legal compliance based on a local synthetic demo.

Close with: “Could the practice's system owner help us describe one administrative handoff using only fictional fields?” A meaningful test requires correct identity linkage, a rejected conflict, visible partial state, and a documented way for staff to resolve it.

Your notes

Administrative handoff, supported system/version, identity key, permitted fields, approval owner, and conflict-resolution rule.

_____

_____

_____

CHAPTER 09 · 12 CHECKPOINTS

Retail and inventory: count once, apply the receipt once

The business story

A delivery is counted, recorded, added to stock, and reflected in availability. Copying the quantity into several places is one problem; adding the same delivery twice is another. This example is valuable because it demonstrates a business effect, not just copied text.

App	What it owns	Proof to show
Receiving Desk	Receipt, purchase order, supplier, item, counted quantity, and unit cost.	An arrival has a stable receipt reference.
Stock Ledger	Stock by SKU, receipt effects, location, and reserved quantity.	Replaying the same receipt does not add another 12 units.
Online Catalog	Local item availability derived from saved stock and reservations.	12 on hand with 2 reserved leaves 10 available.

- [ ] Ask whether staff use supplier item codes, internal SKUs, barcodes, or all three.
- [ ] Ask who owns counted quantity and who owns reserved quantity.
- [ ] Ask how split deliveries, returns, damaged goods, and corrected receipts work.
- [ ] Ask whether stock updates are additive movements or absolute snapshots; confusing these causes major errors.
- [ ] Ask how the existing process detects a repeated import.

Demonstrate it

- [ ] Follow The Works chapter 15. Show the manual forms and their 7, 6, and 5 field positions.
- [ ] Explain that a manual catalog entry does not perform the workflow's stock derivation. The after route demonstrates the carried-forward ledger result.
- [ ] Reset and populate `REC-088`, SKU `FLT-20`, 12 counted units, 2 reserved units.
- [ ] Open Stock Ledger and Online Catalog. Identify 12 on hand and 10 available.
- [ ] Click **Replay the delivery**. Confirm stock did not become 24.

- [ ] Enter a genuinely new receipt reference `REC-089` with the same SKU and 12 more units. Confirm 24 on hand and 22 available.
- [ ] Demonstrate the held correction from The Works if the owner asks about changing an already-applied receipt. Explain that the receiving record may already have changed when the downstream ledger refuses an inconsistent application.

Say: “The receipt ID represents an arrival. The SKU represents the item. We need both. Two deliveries of the same SKU are legitimate; one delivery imported twice must not create twice the stock.”

Next step

Ask for a fictional receiving sheet and the system's supported stock-adjustment model. A real store may require warehouse locations, variants, returns, reservation events, and transaction rules not covered by this small demo. WooCommerce is a researched candidate; this Online Catalog has not published to a real store.

Success means correct stock after a new receipt, unchanged stock after a replay, a held unknown SKU or inconsistent correction, and availability that agrees with the authoritative stock/reservation system.

Your notes

SKU/receipt identities, stock versus movement semantics, reservation owner, correction process, and first receiving example.

_____

_____

_____

CHAPTER 10 · 12 CHECKPOINTS

Independent sellers: one item, prepared listing, one sale

The business story

A person selling used goods types a title, condition, dimensions, weight, and price into inventory notes, a listing, and packing paperwork. A one-off item also must not be sold twice.

App	What it owns	Proof to show
Item Register	Item identity, description, condition, price, physical details, and stock.	`USED-072` identifies one actual demo item.
Listing Desk	A local listing draft using the shared item facts.	The condition and dimensions match the register.
Sales and Packing Desk	Sale reference, buyer/address on completion, and packing information.	An identified sale changes stock once and prepares matching paperwork.

- [ ] Ask how the seller distinguishes two similar items.
- [ ] Ask which marketplaces or channels they use and whether their accounts have approved API access.
- [ ] Ask which fields they repeatedly copy and which must vary by marketplace.
- [ ] Ask where availability is authoritative and how quickly another listing needs to be withdrawn after a sale.
- [ ] Ask who approves publication, selling price, shipping service, and refunds.

Demonstrate it

- [ ] Follow The Works chapter 16. Show the 7, 7, and 7 manual field positions, then the nine shared intake facts.
- [ ] Populate the sample side table and inspect the three saved records.
- [ ] Click **Record a sale**. Review the sample buyer, delivery address, and `SALE-019` before submitting.
- [ ] Show stock moving from one to zero and the local packing document using the sale details.
- [ ] Repeat the same identified sale and explain why it does not subtract stock twice.
- [ ] Try the second distinct sale described in The Works. Show the oversell rejection.

- [ ] Say: “This is a local listing draft and packing document. It has not published an eBay listing, charged a buyer, or bought postage.”

The real marketplace conversation

Different marketplaces have different fields, account entitlements, authorization, and publication rules. A developer key is one part of access, not a universal pass. eBay's developer and sandbox documentation is a research starting point; no eBay approval has been obtained for this showcase. Do not describe a local grant button as vendor consent.

Close with: “Could we take one fictional item through your current inventory notes and listing fields, then define exactly what should happen when it sells?” Include publication and purchase approvals explicitly in any real scope. Do not imply the demo already synchronizes competing marketplace sales atomically.

Your notes

Channels, item ID, repeated listing fields, stock authority, sale event identity, publication approval, and packing requirements.

_____

_____

_____

Property operations: request, contractor, owner, invoice match

The business story

A tenant reports a repair. The office needs a property-linked request, a contractor's work item, and an owner-facing update. Later, the invoice must match the job and the approved amount.

App	What it owns	Proof to show
Property Desk	Property/unit, tenant contact, issue, and ticket.	`FIX-016` remains linked to `PROP-03`, unit `2B`.
Contractor Board	Issue, contact, access window, and assigned contractor.	The contractor-facing work record carries the correct location and access facts.
Owner Report Desk	Job summary, contractor, approved estimate, and invoice-match outcome.	An amount or reference mismatch is held for a person.

- [ ] Ask where requests arrive and who decides urgency, assignment, and access.
- [ ] Ask which information a contractor may receive and which belongs only in internal notes.
- [ ] Ask how job references appear on invoices and what happens when they are missing.
- [ ] Ask who approves estimates and any increase, and where that approval is recorded.
- [ ] Ask whether owner updates are copied manually or generated from the repair record.

Demonstrate it

- [ ] Follow The Works chapter 17. Show 6, 8, and 6 manual field positions, then the nine-field intake.
- [ ] Populate the repair and inspect all three destinations.
- [ ] Click **Match a contractor invoice**. Try a mismatched job reference and show the hold.
- [ ] Try a $200 invoice against the $180 approved estimate. Show that the mismatch prevents the match event from completing.
- [ ] Enter the documented correct job reference and $180 amount. Inspect the successful result.

- [ ] Explain: “This matches a local invoice event against the approved facts. It does not pay the contractor or send an owner email. The demo does not have an overage-approval screen.”

Next step

Request a blank work-order layout and a fictional invoice. Define matching tolerances, partial invoices, taxes, change orders, duplicate invoice IDs, and the person responsible for exceptions before proposing a production rule. These details are business decisions, not assumptions the connector should invent.

Success means the right job and location in all destinations, a clear hold for the wrong reference or amount, and a repeatable matching outcome for the same invoice event.

Your notes

Request channel, property/ticket keys, contractor information, estimate authority, invoice matching rules, and escalation owner.

_____

_____

_____

CHAPTER 12 · 11 CHECKPOINTS

Community and events: registration, capacity, badges, attendance

The business story

An organizer has registrations in a sheet, attendance capacity in another place, and badges or check-in records elsewhere. This makes a useful small-business and community example, and it connects directly to the real public Google Sheet reader.

App	What it owns	Proof to show
Registration Desk	Registration reference, attendee name/email, phone, and session.	Source facts and generated defaults are distinguishable.
Session Board	Session/date/capacity and the reservation or waitlist outcome.	A full session produces a waitlist outcome.
Badge and Attendance Desk	Badge facts, reservation status, and attendance event.	A waitlisted attendee cannot simply be checked in as reserved.

- [ ] Ask how attendees register and whether the spreadsheet is authoritative or only an export.
- [ ] Ask how stable attendee/registration IDs are assigned and how edits or cancellations are handled.
- [ ] Ask who owns session capacity and which people may change it.
- [ ] Ask whether accessibility or contact details should be omitted from particular destinations.
- [ ] Ask what counts as checked in and how repeated scans/clicks are handled.

Demonstrate it

- [ ] Follow The Works chapter 18. Stop any schedule, reset, and demonstrate the 5, 6, and 6 manual field positions versus the eight-field intake.
- [ ] Run the capacity-one example. Avery is reserved; the distinct Taylor registration is waitlisted.
- [ ] Try checking in the waitlisted attendee and show the hold. Use the documented capacity-two practice change, then demonstrate the valid check-in.
- [ ] Explain that this demo takes capacity from the intake. A real shared booking system needs an authoritative, controlled session-capacity record; increasing the field is a practice exercise, not a production approval process.

- [ ] Follow The Works chapter 10 to pull the public Google sample. Point to the original sheet, source rows, column mappings, and the defaults required to fit those sample rows into this event story.
- [ ] If useful, follow chapter 11 to enable a timer, observe a run, and stop it afterward.

Say: “The Google data really came from that public sheet. Its sample names and majors are being mapped into a fictional event example. Generated emails and registration references are demo defaults, not facts supplied by Google.”

Next step

Ask for the actual permitted registration layout and a stable business key. The demo's generated row-number references illustrate a mapping; they are unsuitable as durable identities if a real sheet is freely reordered. A production connector also needs cancellation and correction rules.

Success means correct registration facts, capacity behavior, clear waitlist outcomes, stable identities, and repeated imports that do not create extra attendees.

Your notes

Registration source, stable ID, session authority, capacity owner, waitlist/check-in rules, and destinations for each field.

_____

_____

_____

Sports and personal reports: show where the facts come from

The business story

A person regularly checks a public data source, copies results into a worksheet, adds an event to a calendar, and prepares a summary. The same pattern appears in many permitted reporting tasks: source retrieval, filtering, calculation, a dated result, and a destination someone can use.

App	What it owns	Proof to show
Sports Worksheet	Event/driver facts, counted laps, and calculated best recorded lap.	The values trace to fetched source records.
Schedule Board	The historical event date and display-timezone information.	Event time and retrieval time are different facts.
Report Inbox	Local recipient details and the prepared briefing.	It is saved locally; no internet email has been delivered.

- [ ] Ask which facts the person checks, how often, and why the report matters.
- [ ] Ask whether historical, delayed, or live information is actually needed.
- [ ] Ask what should happen when data is missing, corrected, late, or unavailable.
- [ ] Ask who receives the result and whether they need a file, local view, message, or calendar entry.
- [ ] Ask whether every reader needs a fresh fetch or whether one permitted shared fetch could serve several reports.

Demonstrate it

- [ ] Follow The Works chapter 12. Start at **Sources & connections → Fetch historical F1 data**.
- [ ] Show the three real requests for session, driver, and lap data, using historical session `9158` and driver `1`.
- [ ] Identify the fetched driver/session facts, best positive recorded lap duration, lap count, source URL, and retrieval timestamp.
- [ ] Review the recipient and delivery address. These are local demonstration defaults, not personal information fetched from the sports provider.

- [ ] Populate and open the three destination records. Export the calendar or report if useful.
- [ ] Explain that the best recorded lap in these returned driver records is not a championship result, a race-win determination, or a guarantee that all possible data is present.
- [ ] If using an offline replay, show its original retrieval date and replay label.

Cost and scale explanation

The implemented connector uses OpenF1's public historical interface. Its real-time service is a separate offering; this station does not use it. Check the provider's current documentation before changing the use case. The existing research is linked in chapter 16.

Three requests every five minutes all day would be `3 × 288 = 864` requests per day before retries. This is arithmetic, not a quota, permission to send that many requests, or a measured capacity claim. The demo has one configured source timer and one report recipient. A larger service needs source-specific limits, caching, retry control, and measured load on owned systems.

Close with: “Which report do you manually rebuild, and what source are you permitted to read?” Do not assume that a page visible in a browser provides an approved automation interface.

Your notes

Source, fields, freshness requirement, calculation definition, intended audience, update frequency, and permitted access path.

_____

_____

_____

CHAPTER 14 · 19 CHECKPOINTS

The tools conversation: messy files, sheets, PDFs, and scans

CSV and Excel workbooks

Owner language: “We preserve the source, find the real table, clean supported formatting problems, and flag ambiguity for a person. We do not need you to make the sheet beautiful first.”

Show **Sources & connections → Clean & map** on an ugly generated workbook. Explain the title rows, blank row, header, whitespace, duplicate, malformed email, and ambiguous date. Show the untouched cells beside cleaned values. Open the actual XLSX in LibreOffice using The Works chapters 07–08.

- [ ] Ask who creates the file, which worksheet matters, whether headings move, and whether it contains formulas or merged layouts.
- [ ] Explain the current boundary: CSV/TSV and the first worksheet of XLSX, with the supported table-cleanup rules. This is not an arbitrary spreadsheet formula engine or a promise to understand every workbook layout.
- [ ] Show a ready row and a held row. Explain why silently guessing a date or identifier is worse than asking for review.
- [ ] Show **Populate all ready rows** only after checking the ready/held counts. The review flow is documented in The Works.
- [ ] Ask for a permitted messy sample and define expected clean outputs before estimating a new parser.

Google Sheets

Owner language: “A shared sheet can be the source. We can show which rows came from it and where they landed.” Show the public sample URL, source table, mapping, three destinations, and optional timer.

- [ ] Distinguish a public CSV read from private Google OAuth. The current implementation supports the public route, not private consent or writing back into a sheet.
- [ ] Ask whether the real sheet is public or private, which tab/range is allowed, and who owns access.
- [ ] Ask for a stable row ID and the policy for corrected, deleted, or reordered rows.
- [ ] Explain that current demo defaults fill absent event facts visibly; a real process needs approved default rules or required input.
- [ ] Say “we would need to implement and verify the authorized private connection” when that is the actual requirement.

Text PDFs and populated paperwork

Owner language: “If the input follows a known labelled form, we can extract the fields for review. If the output is a known template, we can put the reviewed facts into it.” These are two different tasks: reading a document and creating one.

- [ ] Upload `practice_files/hvac-intake.pdf` as in The Works chapter 09. Point to the extracted labels and values.
- [ ] Explain that Poppler extracts available text; the mapping recognizes the supported labels. A complex or unrelated PDF may need another parser or template.
- [ ] Open a populated destination document and compare it with the saved record.
- [ ] Ask which version of the real form is authoritative, which repeated fields can be filled, and which fields require a signature or decision.
- [ ] Do not call a generated PDF an electronic filing, signed agreement, or completed payment.

Scans and OCR

Owner language: “A scan starts as pixels. OCR proposes text; the review step is where we check whether the letters and numbers are right.”

- [ ] Upload the supplied HVAC scan. Identify Tesseract as the local OCR tool and inspect the proposed fields.
- [ ] Explain the demo's input boundary: up to 8 MB per upload and OCR of the first three PDF pages. Image quality, handwriting, layout, and label wording can affect extraction.
- [ ] Ask whether real documents are typed, photographed, skewed, handwritten, or multi-page, then test representative permitted samples.
- [ ] Never promise perfect OCR. Define which fields require exact verification before any downstream action.

Local tools versus installed commercial software

LibreOffice is available for opening the practice spreadsheets. The automation reads supported XLSX files locally without requiring Microsoft Excel automation or a paid AI API. That does not establish compatibility with every Excel macro, workbook feature, or enterprise add-in. Demonstrate the files you actually support and name the additional features that would need investigation.

Your notes

The owner's file formats, layout variations, private/public sources, stable identifiers, extraction risks, and representative samples to test.

_____

_____

_____

CHAPTER 15 · 23 CHECKPOINTS

The process conversation: maps, APIs, records, timers, and proof

Field mapping

Owner language: “Different apps call the same fact different things. We translate the names and formats, then check the saved result.” Show **Preview field mapping** and **Under the hood → The translation layer**. Avery's name becomes `display_name`, `contact_name`, or `addressee` depending on the destination.

- [ ] Ask which fields are required and which system is authoritative if values differ.
- [ ] Separate renaming, formatting, calculation, and business approval. They have different failure modes.
- [ ] Point to visible source/default provenance instead of treating every populated field as a fetched fact.

APIs and connectors

Owner language: “An API is the supported door software provides for another program. We need the right door, the right permission, and the right record format.”

- [ ] Open an actual POST request and GET read-back in **Under the hood**.
- [ ] Point to the destination record ID and the value comparison. An HTTP success response alone is less proof than inspecting what was saved.
- [ ] Explain that a connector handles field translation, credentials, matching keys, errors, and provider-specific rules; possession of a key does not implement those parts.
- [ ] If no supported API exists, investigate permitted exports/imports or other authorized interfaces. Browser automation may be an option after review, but this demo does not prove universal screen automation or access to closed software.

Separate records and databases

The 24 workplaces each have their own SQLite database. The coordinator stores jobs, source references, and traces separately. This makes it possible to inspect actual destination state and demonstrate partial completion. It does not reproduce every operational characteristic of 24 commercial SaaS products.

- [ ] Open the three workplace records and show their different IDs.
- [ ] Explain why a customer ID, job ID, and operation ID serve different purposes.

- [ ] Use the duplicate receipt or sale example to explain why a retry must not repeat a business effect.
- [ ] Describe a completed run as a historical snapshot. A later record revision can legitimately differ from it.

Queue, retry, and recovery

Owner language: “If the third destination fails, the first two saves remain visible. We restore access and continue from the saved step.” This is recovery across separate systems, not an all-or-nothing transaction across arbitrary vendors.

- [ ] Show a local 403 after access revocation, or a 503 after taking a destination offline.
- [ ] Show the partial state, restore the condition, and click **Resume from saved step**.
- [ ] Explain that business conflicts such as a double booking need corrected input or a person; repeatedly retrying the same conflict does not fix the business decision.
- [ ] Discuss how a real system would detect and reconcile a response lost after the provider already saved the record. The local demo's identifiers are evidence for its own implementation, not a guarantee about every vendor.

Scheduling and source freshness

Owner language: “The timer re-reads a selected source while the server is running. It is polling; it is not a notification sent by the vendor.”

- [ ] Show the source and interval before enabling the schedule.
- [ ] Explain the current limits: one configured source, minimum one minute, up to 100 clean source rows processed per tick, and no execution while the local server is stopped.
- [ ] Distinguish a source's event time from the time the connector fetched it.
- [ ] Stop the schedule after the demonstration. The persisted opt-in can resume after a restart if left enabled.

Exports, screenshots, diagrams, and automated tests

PDF gives a readable document; XLSX/CSV give data for further work; ICS creates a calendar file; the evidence ZIP packages the run's evidence. None of these by itself proves email delivery, a signed filing, a marketplace publication, or payment. Inspect the exported contents and explain the recipient's next action.

The before/after gallery contains actual browser captures with metadata and hashes. A hash helps identify a file; it does not independently certify the truth of every claim about it. The retained captures show the recorded demonstration, not necessarily the latest live run.

Archi diagrams explain how components and workflows connect. They are maps, not execution engines. Playwright drives the browser for repeatable checks and captures. Application tests exercise business rules and failures. Neither a diagram nor a screenshot replaces inspecting the actual saved records.

- [ ] Open the workflow diagram, then the real records it describes.
- [ ] Open an export and point out its scope.
- [ ] Show the evidence manifest if someone asks when a screenshot was captured.
- [ ] Explain that tests ran against the stated local demo; vendor compatibility requires separate connection tests.

Your notes

Which mechanisms matter to this owner, the proof for each, and the technical questions that still require investigation.

_____

_____

_____

CHAPTER 16 · 7 CHECKPOINTS

Real access: what is connected, what needs approval

Use three accurate categories

Category	Present state	Appropriate language
Implemented local connections	24 fictional workplace apps, real local HTTP requests, independent saved records.	“These apps are ours, and this transfer actually runs.”
Implemented public reads	Google public sample CSV and OpenF1 historical requests, with labelled recordings available.	“This response came from this provider at this time,” or “this is the retained replay.”
Researched candidates	Baserow, ERPNext/Frappe, docassemble, OpenEMR, WooCommerce, eBay, Nextcloud, Mailpit, and other documented options.	“This has a documented interface worth evaluating; it is not connected in this station.”

How a real permission story works

“First we identify the account owner and the operations needed. Then we check the provider's supported interface and account eligibility. The owner or administrator follows the provider's access process. We configure the approved credentials privately, test an allowed operation, test a rejected operation where appropriate, and read back the result. Only then can we say that connection works.”

A developer account, application registration, provider approval, user consent, issued credential, permitted scope, and successful API operation are separate milestones. Some systems need only a scoped key in an owned account; others require an application review or OAuth flow. Do not invent an application or approval requirement before inspecting the actual provider.

- [ ] Record the exact provider, product/version, account plan, and needed read/write operations.
- [ ] Identify who can authorize those operations. An employee who can view a screen may not be able to grant integration access.
- [ ] Use a supported test environment and fictional data where available.
- [ ] Request only the operations needed for the scoped workflow.
- [ ] Keep secret values out of presentation slides, screenshots, browser notes, and chat messages.
- [ ] Collect redacted evidence of the grant and actual saved test result after they occur.
- [ ] Explain revocation and what the operator will see when access is lost.

Source library and candidate talking points

These links are the source library for follow-up investigation. Provider interfaces and entitlements can change. The project's research snapshot is dated September 23, 2026; verify the selected account's current requirements before quoting a production commitment.

Candidate	Why it is relevant	Question before promising a connection
Baserow API	Structured table data and restricted API access.	Which tables and operations may the integration use?
Frappe REST API / ERPNext	Business documents such as work and invoice records.	Which document types, mandatory fields, and user permissions apply?
docassemble API	Guided interviews and document variables.	Who authors and approves the interview and document templates?
OpenEMR API documentation	Administrative healthcare interfaces.	Which deployed version, client setup, scopes, and approved environment are available?
WooCommerce REST API	Store records through a supported interface.	Which store, credentials, product model, and stock operations apply?
eBay developer setup	Marketplace account and application setup.	Has the account actually received the needed access and test-user authorization?
Nextcloud WebDAV	Files and folders as supported destinations.	Which account, folders, and file operations are permitted?
Mailpit	Inspecting local test email.	Is the goal local message proof or actual external delivery?
Google Sheets quickstart	Official sample and authenticated API setup reference.	Public export or private authorized access, and which stable row identifiers?
OpenF1 documentation	The historical sports source used by this demo.	Which endpoints, freshness, current limits, and permitted use are needed?
LibreOffice	Local spreadsheet/document work.	Do the actual files open correctly and use supported features?

The exact candidate status and earlier provider-limit research live in **docs/RESEARCH.md**. The station's verified provider request metadata is in **fixtures/public/verification.json**. Treat the candidate list as a research menu, not a row of completed integrations.

Your notes

Provider, account owner, supported operations, test environment, approval state, redacted proof still needed, and current documentation checked.

_____

_____

_____

Free demonstration, real operating costs, and scale

The straightforward cost answer

“This demonstration has no required paid subscription, paid API call, or paid AI service. It uses the computer and internet connection already available. A production deployment has its own workload, hosting, vendor plans, maintenance, and support choices; those need to be priced for the actual scope.”

Free software does not mean zero human effort or zero electricity/storage. A free API tier does not mean unlimited calls, every endpoint, or guaranteed availability. An existing customer's paid software is a separate expense from your connector. Distinguish these rather than saying everything will always cost nothing.

- [ ] State which costs are currently zero: required demo service charges.
- [ ] State what is assumed already available: the present machine and internet connection.
- [ ] Identify new costs only when a proposed real deployment requires them: hosting, vendor access, delivery, transactions, storage, monitoring, support, or implementation.
- [ ] Verify current provider entitlements and limits for the actual selected account before estimating operating costs.
- [ ] Avoid signing up for a paid trial to make a supposedly free presentation work.

What scale means in this station

The current engine uses three workers, separate local SQLite stores, persisted jobs/traces, and one optional source timer. The demonstration verifies correctness for its tested examples. It does not establish a production user count, guaranteed uptime, or requests-per-second figure for a larger service.

Say: “We can explain how we would scale this, but a capacity claim needs a measured workload and the real provider limits.” A design discussion can cover shared caching, queues, rate limiting, controlled retries, database choices, worker capacity, and monitoring. Label these as production design choices when they are not implemented in the showcase.

Questions that produce an honest capacity estimate

- [ ] How many events arrive per hour, and what is the largest burst?
- [ ] How many records and requests does each event produce?
- [ ] How long can a normal event wait? What is the acceptable recovery time after an outage?

- [ ] Which provider has the tightest rate, payload, concurrency, or account limit?
- [ ] Which work can share cached source data, and what freshness is acceptable?
- [ ] How many exceptions require a person, and who will work that queue?
- [ ] What data must be retained, for how long, and how will backups/restores be tested?

Measure throughput and failures against owned local systems or permitted test infrastructure. Do not load-test a public free API simply because its endpoints are readable. Report the machine, fixture size, concurrency, latency, error/retry rate, and limits alongside any benchmark.

Your notes

Volume, peak burst, freshness, service expectations, provider limits, deployment choices, measured facts, and costs still unknown.

_____

_____

_____

Objections and practical responses

“We already pay for software.”

“Good. Which handoff between the tools still makes someone copy information? If your existing products already do it reliably, that is the first thing to check.” Ask about included integrations before proposing custom work. The value may be configuration, cleanup, or nothing at all; do not invent a need for new code.

“My spreadsheet is too messy.”

“Let's look at one permitted sample. This example shows title rows, extra spaces, duplicates, and values that need review. We can demonstrate which problems the current rules handle and identify what your sheet needs.” Open the ugly workbook and its held row. Do not promise automatic understanding of every layout.

“Can't ChatGPT or a macro do that?”

“A macro or another tool may handle part of it. The question is what happens to the whole handoff: the identifiers, permissions, saved records, repeated runs, and failures. We can choose the smallest reliable method that meets those requirements.” Do not claim a paid model is required; this demo uses deterministic code and local tools.

“What if it gets the data wrong?”

“Let's define the fields that must be checked, the cases that should stop, and who reviews them. Here is a flagged source row and here is the saved read-back.” Explain that verification reduces particular failure modes; it does not prove every business fact is true merely because it was copied correctly.

“What if it crashes halfway through?”

“Here is a deliberately failed third handoff. These earlier records are saved. The run shows the failure and resumes after the problem is resolved.” Show it. Discuss backup and reconciliation requirements for the actual deployment rather than claiming an impossible universal guarantee.

“Can you connect our specific product?”

“Let's check the supported interface, your account's access, and the exact operations. A product name by itself is not enough to promise the connector.” Record the version, plan, account owner, fields, and read/write needs. If only an export is available, explain what that can and cannot automate.

“Can you just use my login?”

“The setup should use the provider's approved integration access and the operations this workflow needs. The account owner can complete that setup privately. I do not need a password in this meeting.” The actual provider may use a token, OAuth, service account, or another documented mechanism; determine which applies.

“Is this actually free?”

“The showcase runs with no required service charges on this machine. Implementation, support, hosting, and your vendor's access terms are separate decisions for a real deployment.” If you have not priced those decisions, say so. Do not give an invented quote to preserve momentum.

“Will it replace my staff?”

“The example removes repeated handling and keeps review decisions visible. Let's measure what work changes and decide where the recovered time is useful.” Do not promise staffing outcomes from a software demonstration.

“I need it to work with absolutely everything.”

“Let's list the systems and the specific handoffs, then verify each available interface. We can show a broad set of patterns here; each real connection still needs its own access, mapping, and tests.” Offer a concrete connector inventory, not a universal compatibility claim.

“Can we start with one process?”

“Yes. Let's choose one complete path with a known source, named destinations, matching rules, a review owner, and an acceptance test.” Use the scope worksheet in chapter 20. A bounded process can still be a complete working delivery.

- [ ] Practice responding to the objection, then asking a useful follow-up question.
- [ ] Open the relevant proof when it exists.
- [ ] Write unknowns down rather than improvising a capability claim.
- [ ] Accept that a business may already have a good solution or may not have a worthwhile automation opportunity.

Your notes

Objections I heard, my concise responses, evidence that answered them, and unanswered questions to investigate.

_____

_____

_____

The senior-team roles across the SDLC

Explain the responsibilities, not invented biographies

The project includes a team charter. Use these roles as accountable responsibilities in a delivery process. One person may hold several roles in a small engagement. The owner should know who decides, who implements, who verifies, and who supports the result; titles alone do not prove quality.

Role	Core responsibility	Concrete work product	Question this role must answer
Workflow/product lead	Understand the business process and acceptance conditions.	Before/after map, scope, priority, measurable outcome.	Are we solving the owner's actual problem?
Solution architect	Define system boundaries, ownership, and failure behavior.	Component diagram, data contract, recovery design.	What happens when only part of the work succeeds?
Integration/backend engineer	Implement supported reads/writes, matching keys, and resumable jobs.	Working connectors, API traces, persistence.	Do the correct records get saved once?
Data/document engineer	Handle messy inputs, normalization, mappings, and templates.	Fixtures, parsers, review rules, populated documents.	Are facts extracted accurately and defaults visible?
Frontend/experience engineer	Make the operator's work understandable.	Clickable screens, review/exception UI, accessible instructions.	Can the person finish and recover the task?
Domain reviewer	Validate the real business rules and approval points.	Approved examples and exception rules.	Are calculations and decisions appropriate for this process?
Access/security engineer	Plan authorized access and controlled handling of credentials/data.	Scoped access setup, redacted proof, revocation behavior.	Can the connector do only what was authorized?
QA/test engineer	Verify normal, duplicate, invalid, and interrupted behavior.	Reproducible checks, before/after evidence, defect results.	What evidence demonstrates acceptance?
Operations/reliability engineer	Plan running, monitoring, backup, recovery, and support.	Runbook, restore exercise, ownership/escalation plan.	Who notices a problem and gets work moving again?
Documentation/demo lead	Make the delivered system teachable and reviewable.	Operator book, conversation guide, diagrams, handover.	Can the owner use and explain the system without its author?

The SDLC checkpoints you can explain to an owner

- [ ] **Discover:** confirm the manual path, volume, exceptions, and owner of each fact.
- [ ] **Define:** agree on the source, destinations, matching keys, approvals, and measurable acceptance tests.
- [ ] **Design:** draw the handoffs and failure/recovery path; inspect real access requirements.
- [ ] **Build:** implement the agreed complete workflow with permitted sample data and visible operator controls.
- [ ] **Verify:** exercise success, duplicate, correction, malformed input, access loss, outage, and business conflict as applicable.
- [ ] **Demonstrate:** let the intended user operate it and inspect saved outputs, not merely watch a developer click.
- [ ] **Deliver:** provide startup, shutdown, recovery, backup, access ownership, and support responsibilities.
- [ ] **Operate:** monitor the actual workload and revisit changes in sources, vendor interfaces, and business rules.

The current showcase demonstrates these disciplines for local synthetic workflows. It does not mean every researched vendor has completed every checkpoint.

Your notes

Who will own each responsibility in the real engagement, which roles I can cover, and where a domain expert or account owner is required.

_____

_____

_____

CHAPTER 20 · 7 CHECKPOINTS

Scope a complete first business workflow

Use a scope statement the owner can check

“When [trigger] occurs, read [permitted source], identify the record by [stable key], validate [required facts], obtain [human approval if required], save [named fields] to [named destinations], verify [saved values], and route [defined failures] to [responsible person].”

Then name exclusions explicitly because they change the outcome: for example, creating an invoice draft may be included while payment collection and sending it to customers require separately agreed behavior. This is a scope boundary for a real proposal, not a claim that the demo already supports those actions.

The handoff agreement

Item	Fill this in before making a delivery commitment
Trigger and volume	Event, frequency, normal count, peak burst.
Source and permission	Exact file/app/account and approved access.
Destinations	Named products, environments, and supported operations.
Matching keys	Customer, job, item, receipt, and event identities.
Fields and ownership	Required fields, formats, authoritative source, approved defaults.
Decisions	Reviewer, approval condition, and rejection outcome.
Corrections	Update, cancellation, deletion, and duplicate rules.
Failure handling	Retryable versus held conditions, operator visibility, escalation.
Outputs	Saved records, files, reports, notifications, or drafts actually included.
Acceptance	Exact examples and observable outcomes.
Operation	Hosting, credentials, backups, monitoring, support, and change ownership.
Cost	Implementation effort, selected service charges, and ongoing support assumptions.

Acceptance examples that are stronger than “it works”

- [ ] A valid sample creates the expected records with all agreed fields correct and linked.

- [ ] Replaying the same identified event does not create a second receipt effect, sale, or charge.
- [ ] A required missing or ambiguous value is visibly held before the disallowed continuation.
- [ ] A destination failure leaves understandable partial state and can recover without duplicating completed effects.
- [ ] A corrected record follows the agreed update policy and preserves identity.
- [ ] The intended operator can start, inspect, resolve an exception, export evidence, and stop the system using the handover guide.
- [ ] The owner can identify what is outside the agreed path and who handles it.

A concrete next-step question

“Can we agree on one process, one fictional or permitted redacted sample, the account owner who can verify access, and the exact result you would need to see?”

Do not request a pile of passwords or a whole production database to keep the conversation moving. A blank form, a few fabricated rows, and the named system/version are often enough to make the next technical investigation concrete.

Your notes

Draft the complete scope sentence, included outputs, explicit boundaries, acceptance examples, owner, and unresolved access questions.

_____

_____

_____

Follow through after the demonstration

A follow-up message you can adapt

This is a writing template for you. Nothing in this book sends a message.

```
Thanks for walking me through [specific process].

My understanding is that [role] receives [source], then copies
[facts] into [destinations]. The main issue you identified was [pain].

We demonstrated [specific local workflow and proof]. We still need
to verify [actual product access / mapping / business rule].

The proposed next step is [one concrete action], owned by [person].
The permitted sample is [blank, fictional, or approved redacted sample].
Success would mean [observable acceptance condition].

Please correct anything I misunderstood about the process or priorities.
```

- [ ] Replace every bracket with a concrete fact from the conversation.
- [ ] Separate what was demonstrated from what still needs investigation.
- [ ] Include the one agreed action, its owner, and the next review time if one was agreed.
- [ ] Include only links or evidence appropriate for the recipient. A `127.0.0.1` link runs on the recipient's own computer and will not expose your local station to them.
- [ ] If sharing a PDF or evidence packet, inspect it first and use only permitted sample information.
- [ ] Record the unresolved questions and the next technical investigation in your own project notes.

Handling requests outside the shown scope

If the owner asks for messaging, payments, filings, private account connections, or publication, describe the specific authorization and implementation that would be needed. Do not perform an external action just because it resembles a local demo button. The next useful artifact is a concrete scope and reviewed example, followed by the actual owner's authorization for the intended real operation.

After your own rehearsal

- [ ] Stop a source timer you enabled.
- [ ] Restore simulated access and service availability for the next presentation.
- [ ] Export this book's progress and notes.
- [ ] Write down the exact sentence or click that confused you. Revisit that part of The Works before the next meeting.
- [ ] Keep claims aligned with the current application and evidence. If a feature changes, update the demonstration script as well.

Your notes

Agreed next action, owner, timing, evidence to share, unresolved questions, and the follow-up draft I will review before sending myself.

_____

_____

_____

CHAPTER 22 · 30 CHECKPOINTS

Your reusable meeting and readiness checklist

Before speaking

- [ ] I know which business/process I am discussing.
- [ ] I can open the station from Dolphin or Kitty without help.
- [ ] I have chosen one primary workflow and one meaningful exception.
- [ ] I can identify real external data, synthetic data, defaults, and replayed data.
- [ ] My chosen source and destinations are ready, and no unwanted timer is running.
- [ ] I can show the manual forms, source file, three saved records, diagram, and relevant export.

During discovery

- [ ] I asked for a recent example and repeated the process back accurately.
- [ ] I captured the trigger, person, source, repeated fields, and destinations.
- [ ] I identified stable matching keys and the owner of each important fact.
- [ ] I asked about review, corrections, duplicate events, and failed handoffs.
- [ ] I learned what outcome the owner cares about and what evidence would support it.

During the proof

- [ ] I showed the repeated manual work and the source values.
- [ ] I explained how review and mapping determine the saved fields.
- [ ] I opened the actual saved records and linked identifiers.
- [ ] I showed the relevant business rule or failure/recovery.
- [ ] I described exports and screenshots accurately.
- [ ] I avoided calling a researched vendor a completed connection.

- [ ] I separated field counts, observations, estimates, and future capacity designs.

Before ending the conversation

- [ ] I asked what differs in their real process.
- [ ] I recorded their software names and the account/approval owner.
- [ ] I distinguished the free showcase from the costs of a real deployment.
- [ ] We identified one useful next step, or agreed there is no current need.
- [ ] I know what sample and acceptance condition are needed next.

My meeting record

Your notes

Business or fictional rehearsal name; date; people/roles; current process; strongest pain; workflow shown; proof observed; differences; actual tools; access owner; baseline/target; unknowns; agreed action; responsible person; next review.

_____

_____

_____

My own readiness test

- [ ] I can explain the whole journey in ordinary language before opening technical details.
- [ ] I can answer “where did this data come from?” by opening source evidence.
- [ ] I can answer “where did it land?” by opening the actual saved record.
- [ ] I can answer “what if it fails?” by demonstrating the recovery path.
- [ ] I can answer “what does free mean?” without implying unlimited production service.
- [ ] I can say “that still needs verification” clearly when a claim is not yet supported.

- [ ] **Book complete:** I can lead an honest, specific conversation, show a complete working process, and define the next useful piece of work.